

Sobrecarga de operadores y compiladores Debug y Release para un campo eléctrico

Issabela del Pilar Ortiz Infante

Santiago Talero Parra

Juan Sebastian Manrique Moreno

Problema físico

El problema físico a trabajar es el de dos cargas que generan un campo eléctrico interactuante con unas posiciones y velocidades iniciales. La expresión matemática a trabajar es:

$$\mathbf{F} = m\mathbf{a} \text{ y } \mathbf{F} = q\mathbf{E} \quad \therefore \quad \mathbf{a} = \frac{q\mathbf{E}}{m}$$

Donde $\mathbf{E} = \frac{1}{4\pi\epsilon_0} \frac{q}{r^2} \hat{\mathbf{r}}$, y se aplica el método de Euler.

Archivos contruidos

Se generaron 5 archivos distintos que se abordarán en su estructura y funciones:

- Particle.h
- Particle.cpp
- Utils.h
- Main.cpp
- Makefile

Archivo Particle.h

Guardas de Inclusión

Evita que el archivo se incluya más de una vez durante la compilación, lo cual puede causar errores si la clase o estructura se redefine.

```
#ifndef PARTICLE_H
#define PARTICLE_H

#include <iostream> // Para operaciones de entrada/salida (cout, cin)
#include <fstream>  // Para manejo de archivos (ofstream)
```

Archivo Particle.h

struct Vector2D

Representa un **vector bidimensional**, usado para posiciones, velocidades y campos eléctricos.

En la imagen se visualiza el constructor y la suma.

```
struct Vector2D
{
    double x; // Componente x en metros (m)
    double y; // Componente y en metros (m)

    // Constructor con valores por defecto (0.0 m, 0.0 m)
    Vector2D(double x_ = 0.0, double y_ = 0.0) : x(x_), y(y_) {}

    // Sobrecarga del operador + para suma de vectores
    Vector2D operator+(const Vector2D &other) const
    {
        return Vector2D(x + other.x, y + other.y); // Retorna suma componente a
componente
    }
}
```

Archivo Particle.h

struct Vector2D

Representa un **vector bidimensional**, usado para posiciones, velocidades y campos eléctricos.

En la imagen se visualiza la multiplicación por escalar y la división.

```
// Sobrecarga del operador * para multiplicación por escalar (adimensional)
Vector2D operator*(double scalar) const
{
    return Vector2D(x * scalar, y * scalar); // Escala ambas componentes
}

// Sobrecarga del operador / para división por escalar (adimensional)
Vector2D operator/(double scalar) const
{
    return Vector2D(x / scalar, y / scalar); // Divide ambas componentes
}
```

Archivo Particle.h

struct Vector2D

friend permite que funciones externas accedan a los datos privados/protegidos de una clase o struct.

Así mismo, la sobrecarga de operadores cambia el cómo se comporta un operador estándar (+, *, <<, etc.) **para tipos de datos personalizados**, como la estructura Vector2D.

```
// Sobrecarga del operador << para salida por consola (unidades implícitas)
friend std::ostream &operator<<(std::ostream &os, const Vector2D &v)
{
    os << v.x << " " << v.y; // Formato: "x y"
    return os;
}

// Sobrecarga del operador << para salida a archivo (unidades implícitas)
friend std::ofstream &operator<<(std::ofstream &ofs, const Vector2D &v)
{
    ofs << v.x << " " << v.y; // Formato: "x y"
    return ofs;
}
};
```

Archivo Particle.h

Clase Particle

Atributos:

- mass: Masa de la partícula en kg.
- charge: Carga de la partícula en C (culombios).
- position: Posición actual en el espacio.
- velocity: Velocidad actual.

El método update está definido en Particle.cpp

```
// Clase que modela una partícula física
class Particle
{
public:
    double mass;        // Masa en kilogramos (kg)
    double charge;      // Carga en culombios (C)
    Vector2D position;  // Posición en metros (m)
    Vector2D velocity;  // Velocidad en metros/segundo (m/s)

    // Constructor con inicialización de parámetros
    Particle(double m, double q, Vector2D pos, Vector2D vel)
        : mass(m), charge(q), position(pos), velocity(vel) {}

    // Método para actualizar posición y velocidad bajo un campo eléctrico
    // electricField: Campo eléctrico en voltios/metro (V/m)
    // dt: Paso temporal en segundos (s)
    void update(const Vector2D &electricField, double dt);
};

#endif // Fin del guardia de inclusión
```

```

#include "Particle.h"

// Actualiza el estado de la partícula bajo un campo eléctrico aplicado.
void Particle::update(const Vector2D &electricField, double dt)
{
    // Fuerza eléctrica:  $F = q * E$  (Ley de Lorentz para campos estáticos)
    Vector2D force = electricField * charge;

    // Aceleración:  $a = F/m$  (Segunda Ley de Newton)
    Vector2D acceleration = force / mass;

    // Integración tipo Euler explícito:
    velocity = velocity + acceleration * dt; // Actualiza velocidad [m/s]
    position = position + velocity * dt;    // Actualiza posición [m]
}
```

Particle.cpp

Este método es una forma **simple y rápida** de aproximar el movimiento de objetos bajo fuerzas. Se asume que la aceleración es constante durante un pequeño intervalo dt .

Se utiliza el método de Euler.



```
#ifndef UTILS_H
#define UTILS_H

#include <cmath>      // Para funciones matemáticas (sqrt, M_PI)
#include "Particle.h" // Definición de Particle y Vector2D

// Constantes físicas fundamentales (unidades SI)
const double EPSILON_0 = 8.854187817e-12; // Permitividad del vacío [F/m]
const double ELECTRON_CHARGE = -1.602176634e-19; // Carga del electrón [C]
const double PROTON_CHARGE = 1.602176634e-19; // Carga del protón [C]
const double ELECTRON_MASS = 9.10938356e-31; // Masa del electrón [kg]
const double PROTON_MASS = 1.6726219e-27; // Masa del protón [kg]
```

utils.h

Definimos tanto las constantes como la implementación de *cmath* y *Particle.h*

Constante	Significado	Valor (SI)
EPSILON_0	Permitividad del vacío	$\approx 8.85 \times 10^{-12}$ F/m
ELECTRON_CHARGE	Carga del electrón	$\approx -1.60 \times 10^{-19}$ C
PROTON_CHARGE	Carga del protón	$\approx +1.60 \times 10^{-19}$ C
ELECTRON_MASS	Masa del electrón	$\approx 9.11 \times 10^{-31}$ kg
PROTON_MASS	Masa del protón	$\approx 1.67 \times 10^{-27}$ kg

```

// Calcula el campo eléctrico generado por una partícula cargada.
Vector2D calcularCampo(const Particle &fuente, const Vector2D &destino)
{
    double dx = destino.x - fuente.position.x; // Diferencia en x [m]
    double dy = destino.y - fuente.position.y; // Diferencia en y [m]
    double r2 = dx * dx + dy * dy;              // Distancia al cuadrado [m²]
    double r = std::sqrt(r2);                   // Distancia [m]

    // Evita división por cero si las partículas están muy cerca
    if (r < 1e-12)
        return Vector2D(0.0, 0.0);

    // Ley de Coulomb:  $E = (1/4\pi\epsilon_0) * (q/r^2) * (\text{vector unitario } \hat{r})$ 
    double coef = (1.0 / (4 * M_PI * EPSILON_0)) * (fuente.charge / r2);
    return Vector2D(coef * dx / r, coef * dy / r); // Campo en componentes x/y
}

#endif // UTILS_H

```

utils.h

Función calcularCampo:

Propósito:

Calcular el campo eléctrico que una partícula fuente genera en una posición destino.

Parámetros:

- **fuente:** una instancia de Particle que actúa como fuente del campo.
- **destino:** un Vector2D que representa la posición donde se evalúa el campo.

main.cpp

- iostream, ofstream: para salida en consola y en archivos.
- chrono: para medir cuánto tiempo tarda la simulación.
- Particle.h: define la estructura y lógica de las partículas.
- utils.h: contiene calcularCampo y constantes físicas.



```
#include <iostream> // Para entrada/salida estándar (cout)
#include <fstream>   // Para manejo de archivos (ofstream)
#include <chrono>     // Para medición de tiempo (high_resolution_clock)
#include "Particle.h" // Clase Particle y definiciones de constantes (PROTON_MASS, etc.)
#include "utils.h"    // Funciones auxiliares (calcularCampo, operadores Vector2D)
```

main.cpp

El modelo:

- p1: un **protón** en el origen, en reposo.
- p2: un **electrón** a una distancia típica del radio de Bohr (5.29×10^{-11} m) y con la velocidad orbital típica (2.19×10^6 m/s).

Una órbita atómica muy simplificada.

Así mismo los parámetros de integración, las salidas en archivos de texto y medición del tiempo que toma la simulación completa para fines de benchmarking o análisis de rendimiento.

```
int main()
{
    // Crea partículas: protón (p1) y electrón (p2) con posición/velocidad inicial
    Particle p1(PROTON_MASS, PROTON_CHARGE, {0.0, 0.0}, {0.0, 0.0});
    Particle p2(ELECTRON_MASS, ELECTRON_CHARGE, {5.29e-11, 0.0}, {0.0, 2.19e6});

    double dt = 1e-18; // Paso temporal de simulación (en segundos)
    int steps = 1000; // Número de iteraciones

    // Archivos para guardar trayectorias de las partículas
    std::ofstream archivo1("particula1.txt");
    std::ofstream archivo2("particula2.txt");

    // Inicia medición de tiempo de ejecución
    auto inicio = std::chrono::high_resolution_clock::now();
```

main.cpp

Para cada paso de tiempo:

Calcular el campo eléctrico que p1 genera en la posición de p2 $\rightarrow$ E1

Calcular el campo eléctrico que p2 genera en la posición de p1 $\rightarrow$ E2

Actualizar la velocidad y posición de p1 usando el campo E2

Actualizar la velocidad y posición de p2 usando el campo E1

Mostrar posiciones en consola

Guardar posiciones en archivos

```
// Bucle de simulación
for (int i = 0; i < steps; ++i)
{
    // Calcula campo eléctrico en la posición de cada partícula
    Vector2D E1 = calcularCampo(p1, p2.position);
    Vector2D E2 = calcularCampo(p2, p1.position);

    // Actualiza posición/velocidad de las partículas (F = q*E)
    p1.update(E2, dt);
    p2.update(E1, dt);

    // Muestra posiciones en consola
    std::cout << "Paso " << i << ":\n";
    std::cout << "  Partícula 1: " << p1.position << "\n";
    std::cout << "  Partícula 2: " << p2.position << "\n";

    // Guarda posiciones en archivos
    archivo1 << p1.position << "\n";
    archivo2 << p2.position << "\n";
}
```

main.cpp

Se detiene el proceso de medición temporal así mismo se cierran los archivos.

Muestra el tiempo total de simulación.

```
// Finaliza medición de tiempo y muestra resultados
auto fin = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> duracion = fin - inicio;
archivo1.close();
archivo2.close();

std::cout << "Tiempo total de simulación: " << duracion.count() << " s" << std::endl;

return 0;
}
```



```
# Configuración general
CXX = g++
SRC = main.cpp Particle.cpp
TARGET_DEBUG = simulacion_debug
TARGET_RELEASE = simulacion_release

CXXFLAGS_DEBUG = -g -O0
CXXFLAGS_RELEASE = -O3

N = 100
```

makefile

Definimos las banderas tanto para Debug como para Release:

- -g: incluye información de **depuración** (debug symbols).
- -O0: **sin optimizaciones** del compilador → más lento, pero más fácil de rastrear errores.

Usado para:

- Ver fallos con gdb.
- Estudiar línea por línea lo que hace el programa.



```
# Configuración general
CXX = g++
SRC = main.cpp Particle.cpp
TARGET_DEBUG = simulacion_debug
TARGET_RELEASE = simulacion_release

CXXFLAGS_DEBUG = -g -O0
CXXFLAGS_RELEASE = -O3

N = 100
```

makefile

- -O3: nivel de optimización más alto. El compilador reordena y mejora código agresivamente para que **corra rápido**.

Usado para:

- Producción.
- Benchmarking de velocidad.
- Comparaciones de eficiencia.

N será el número de ejecuciones para comparar rendimientos

```

# Compilación en modo Debug
debug:
    $(CXX) $(CXXFLAGS_DEBUG) $(SRC) -o $(TARGET_DEBUG)

# Compilación en modo Release
release:
    $(CXX) $(CXXFLAGS_RELEASE) $(SRC) -o $(TARGET_RELEASE)

# Limpieza
clean:
    rm -f *.o $(TARGET_DEBUG) $(TARGET_RELEASE) \
        particula1.txt particula2.txt \
        tiempos_debug.txt tiempos_release.txt \
        grafica_tiempos.png trayectorias.png temp_time.txt

run_debug:
    @if [ ! -f $(TARGET_DEBUG) ]; then \
        echo "Compilado Debug no encontrado. Compilando..."; \
        $(MAKE) debug; \
    fi
    ./${TARGET_DEBUG}

run_release:
    @if [ ! -f $(TARGET_RELEASE) ]; then \
        echo "Compilado Release no encontrado. Compilando..."; \
        $(MAKE) release; \
    fi
    ./${TARGET_RELEASE}

```

makefile

- Ordenes de ejecución por consola para compilar en ambos modos.
- Limpieza de archivos binarios y resultados previos.
- Ejecución de Debug y de Release.

```
# Medición de tiempos (extrae el tiempo impreso por el programa)

test_timing: debug release

    @echo "Ejecutando $(N) pruebas en modo Debug..."
    @> tiempos_debug.txt
    @for i in $(seq 1 $(N)); do \
        TIME=$(./$(TARGET_DEBUG) | grep -oP 'Tiempo total de simulación: \K\d+\.\d+'); \
        echo "$$TIME" >> tiempos_debug.txt; \
    done

    @echo "Ejecutando $(N) pruebas en modo Release..."
    @> tiempos_release.txt
    @for i in $(seq 1 $(N)); do \
        TIME=$(./$(TARGET_RELEASE) | grep -oP 'Tiempo total de simulación: \K\d+\.\d+'); \
        echo "$$TIME" >> tiempos_release.txt; \
    done
```

makefile

- La regla test_timing ejecuta **100 veces** el binario en ambos modos y guarda los tiempos en dos archivos:
 - tiempos_debug.txt
 - tiempos_release.txt

¿Cómo captura el tiempo?

Busca en la salida del programa la línea:

Tiempo total de simulación:
X.XXXXXXX s

Y con `grep -oP`, extrae justo el número (X.XXXXXXX).

```

# Generar gráfica con promedios
graph: test_timing
    @if command -v gnuplot >/dev/null 2>&1; then \
        echo "Calculando promedios..."; \
        DEBUG_AVG=$(awk '{ sum += $1 } END { printf "%.6f", sum/NR }' tiempos_debug.txt); \
        \
        RELEASE_AVG=$(awk '{ sum += $1 } END { printf "%.6f", sum/NR }' \
tiempos_release.txt); \
        echo "Debug avg: $$DEBUG_AVG s"; \
        echo "Release avg: $$RELEASE_AVG s"; \
        echo "Generando gráfica con promedios..."; \
        gnuplot -e " \
            set terminal png size 800,600; \
            set output 'grafica_tiempos.png'; \
            set title 'Comparación de Tiempos (Debug vs Release)'; \
            set xlabel 'Ejecución'; \
            set ylabel 'Tiempo (s)'; \
            set key left top; \
            set grid; \
            plot 'tiempos_debug.txt' with lines title 'Debug (avg: $$DEBUG_AVG s)' lc rgb \
'red', \
                'tiempos_release.txt' with lines title 'Release (avg: $$RELEASE_AVG s)' lc \
rgb 'blue'; \
        "; \
        echo "Gráfica generada: grafica_tiempos.png"; \
    else \
        echo "ERROR: gnuplot no está instalado."; \
        exit 1; \
    fi

```

makefile

Con este fragmento dentro del mismo makefile podemos graficar los promedios temporales de ambos modos de compilación.

```
# Graficar trayectorias XY de las dos partículas
plot_positions:
    @echo "Graficando trayectorias de partículas..."
    @gnuplot -e "\
        set terminal png size 800,600; \
        set output 'trayectorias.png'; \
        set title 'Trayectoria de las Partículas'; \
        set xlabel 'x'; \
        set ylabel 'y'; \
        set format x '%.1e'; \
        set format y '%.1e'; \
        set autoscale; \
        set grid; \
        set key top left; \
        plot 'particula1.txt' using 1:2 with points lt rgb 'red' pt 7 ps 0.5 title
'Partícula 1', \
            'particula2.txt' using 1:2 with points lt rgb 'blue' pt 7 ps 0.5 title
'Partícula 2';"
    @echo "Trayectorias graficadas: trayectorias.png"
```

makefile

Con este fragmento dentro del mismo makefile podemos graficar trayectorias XY de las partículas.

Luego la ejecución de los diferentes archivos quedan compactados de la siguiente manera:

Objetivo	Comando
Compilar (Debug)	`make debug`
Compilar (Release)	`make release`
Ejecutar (Debug)	`make run_debug`
Ejecutar (Release)	`make run_release`
Medir tiempos	`make test_timing`
Graficar comparación	`make graph`
Graficar trayectorias	`make plot_positions`
Limpiar archivos	`make clean`

Resultados de una muestra de datos

- Primeras condiciones con Debug y Release.
- Segundas condiciones con Debug y Release.
- Tiempos promedios en el cálculo.

Primeras Condiciones:

```
Particle p1(PROTON_MASS, PROTON_CHARGE, {0.0, 0.0}, {0.0, 0.0});  
Particle p2(ELECTRON_MASS, ELECTRON_CHARGE, {5.29e-11, 0.0}, {0.0, 2.19e6});
```

Debug

0.0430961 s

2.97 ms

Tiempo total de la simulación

Tiempo promedio de la simulación

Release

0.0213519 s

2.61 ms

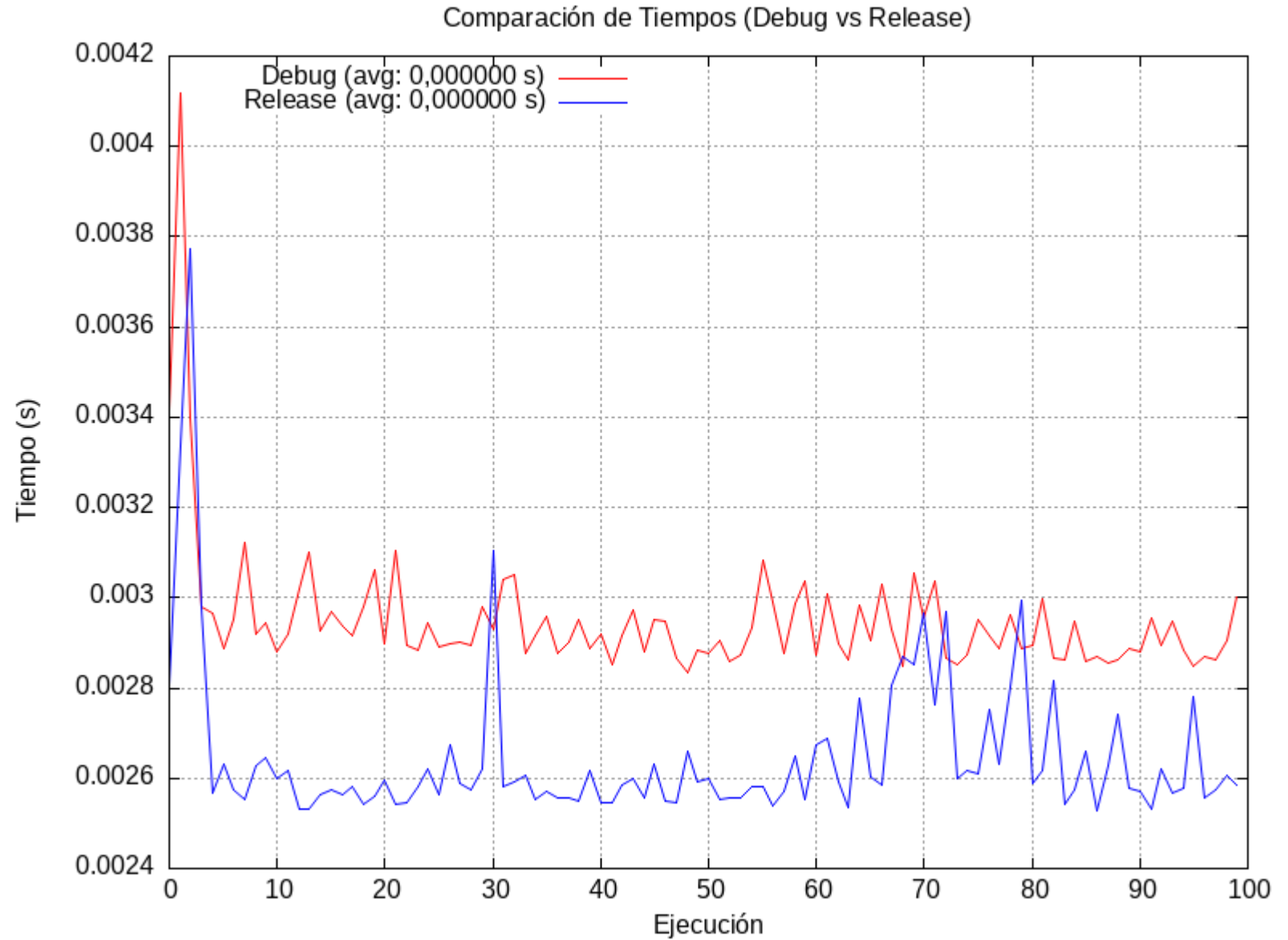
≡ tiempos_debug.txt U

```
pyter/HPC/ParticleSimulation$ make test_timing  
g++ -g -O0 main.cpp Particle.cpp -o simulacion_debug  
g++ -O3 main.cpp Particle.cpp -o simulacion_release  
Ejecutando 100 pruebas en modo Debug...  
Ejecutando 100 pruebas en modo Release...
```

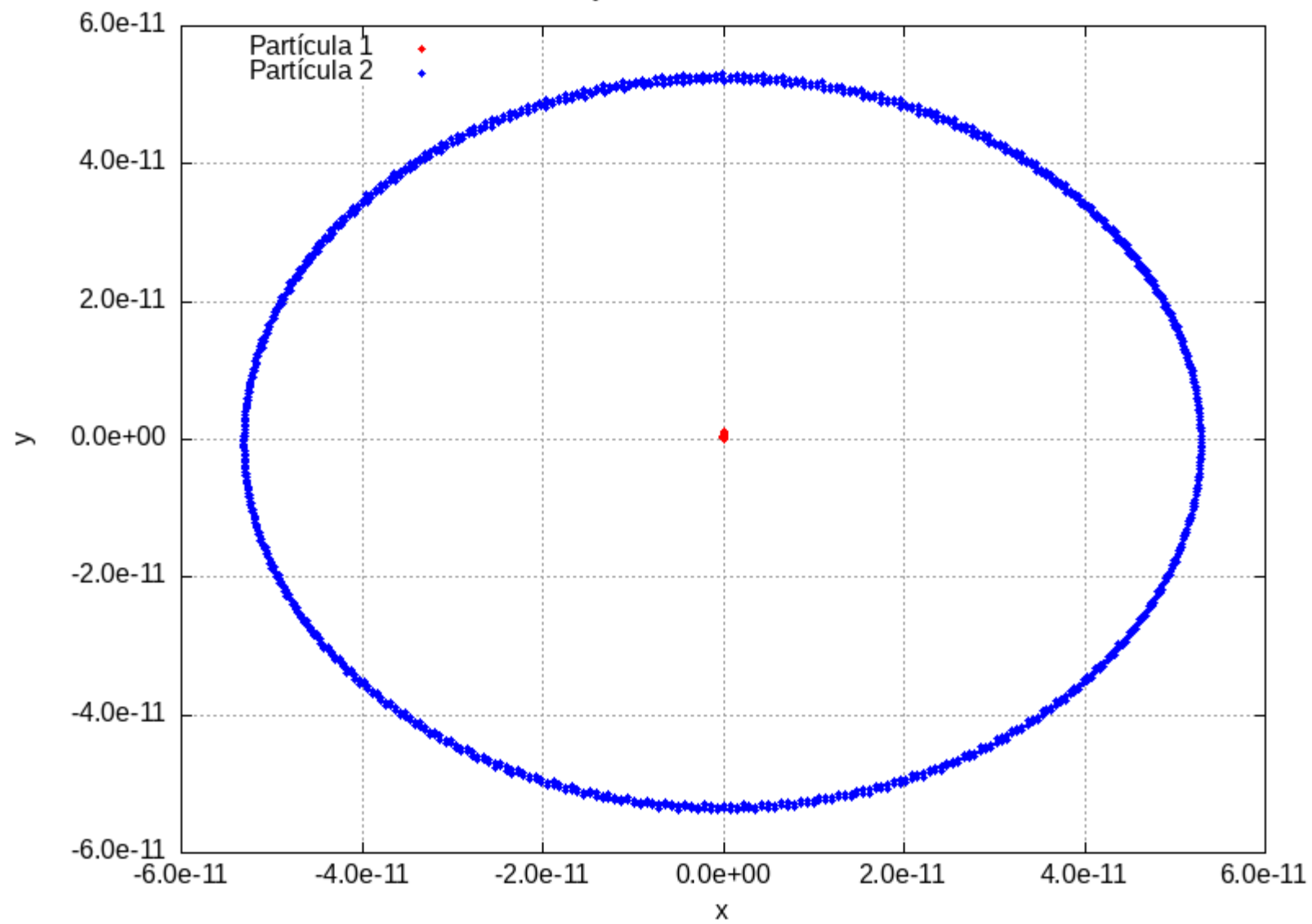
≡ tiempos_release.txt U

Comparamos el tiempo que tarda el programa en ejecutarse 100 veces en dos modos diferentes de compilación:

- Debug:
 - **Promedio constante** alrededor de **~ 0.00297 s**,
 - Tiene **variaciones más grandes** al inicio y en medio de la ejecución.
 - **Código más pesado**
- Release:
 - **Promedio constante** alrededor de **~ 0.00261 s**,
 - Tiene algunas fluctuaciones al inicio, pero luego es **más estable y bajo** que Debug.
 - Eliminamos cosas innecesarias y optimiza todo.



Trayectoria de las Partículas



Segundas Condiciones:

```
Particle p1(PROTON_MASS, PROTON_CHARGE, {5.29e-11, 0.0}, {0.0, 2.19e6});  
Particle p2(ELECTRON_MASS, ELECTRON_CHARGE, {0.0, 0.0}, {0.0, 0.0});
```

Debug

0.036417 s

3.14 ms

Tiempo total de la simulación

Tiempo promedio de la simulación

Release

0.0159822s

2.81 ms

```
≡ tiempos_debug.txt U
```

```
• pyter/HPC/ParticleSimulation$ make test_timing  
g++ -g -O0 main.cpp Particle.cpp -o simulacion_debug  
g++ -O3 main.cpp Particle.cpp -o simulacion_release  
Ejecutando 100 pruebas en modo Debug...  
Ejecutando 100 pruebas en modo Release...
```

```
≡ tiempos_release.txt U
```

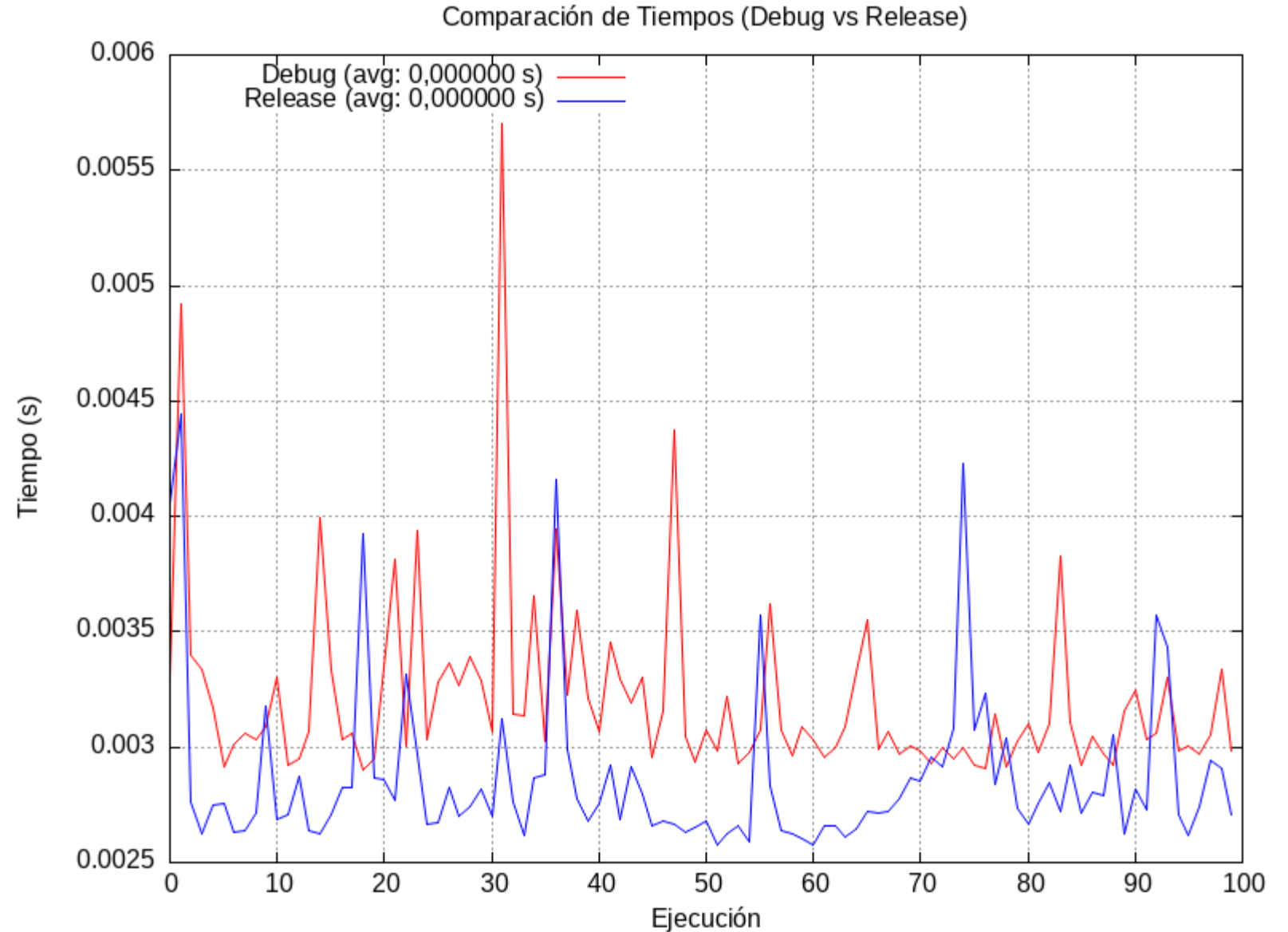
Comparamos el tiempo que tarda el programa en ejecutarse 100 veces en dos modos diferentes de compilación:

● Debug:

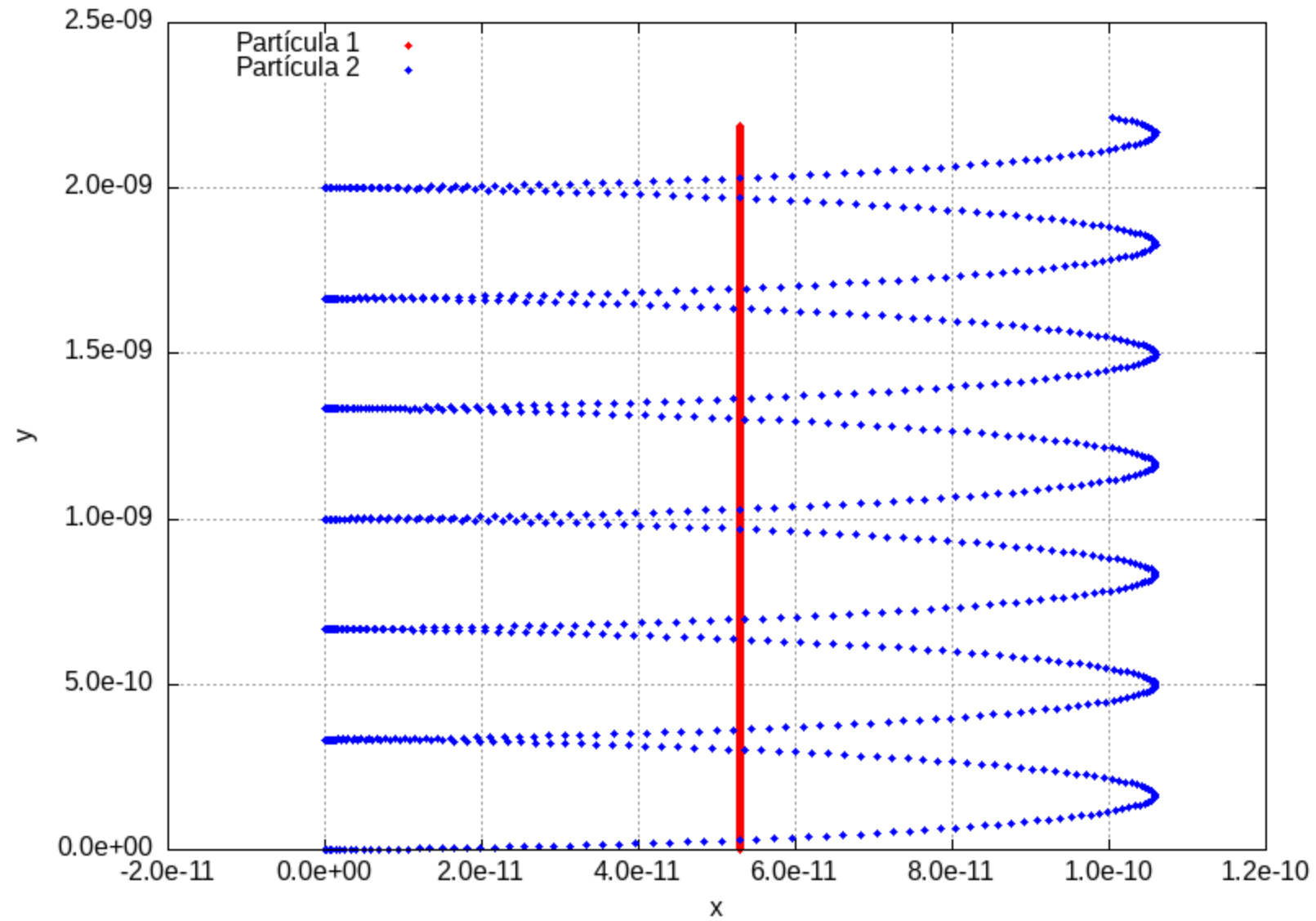
- Presenta **mayor tiempo promedio** de ejecución.
- Se observan **picos frecuentes** de hasta **0.0045–0.0053 s**.
- Tiene menos optimización, por lo que:
- Se ejecutan más instrucciones.
- Se incluyen símbolos de depuración.
- El acceso a memoria no está optimizado.
- Tiende a ser más **sensible al ruido del sistema**.

● Release:

- Corre con **tiempos más bajos y estables**.
- Tiene algunos picos, pero menos pronunciados y menos frecuentes que Debug.
- Se ejecuta con **optimización máxima (-O3)**, lo que reduce:
- Tiempo en ciclos.
- Acceso redundante a memoria.
- Operaciones intermedias innecesarias.



Trayectoria de las Partículas



Posibles optimizaciones

- Particle.h
- Sobrecarga de operadores
- Detalles en Debug y Release

Archivo Particle.h

- El uso de friend para `operator<<` con `std::ofstream` es innecesario si ya está definido para `std::ostream`, ya que `std::ofstream` hereda de `std::ostream`.

```
// Sobrecarga del operador << para salida por consola (unidades implícitas)
friend std::ostream &operator<<(std::ostream &os, const Vector2D &v)
{
    os << v.x << " " << v.y; // Formato: "x y"
    return os;
}

// Sobrecarga del operador << para salida a archivo (unidades implícitas)
friend std::ofstream &operator<<(std::ofstream &ofs, const Vector2D &v)
{
    ofs << v.x << " " << v.y; // Formato: "x y"
    return ofs;
}

};
```

Particle.cpp

```
#include "Particle.h"

void Particle::update(const Vector2D &electricField, double dt) {
    // Método de integración de Runge-Kutta de 2do orden (RK2 / Heun)

    // k1: aceleración y velocidad iniciales
    Vector2D acceleration_k1 = electricField * (charge / mass);
    Vector2D velocity_k1 = velocity;
    Vector2D position_k1 = position;

    // Estimación provisional (Euler)
    Vector2D velocity_temp = velocity + acceleration_k1 * dt;
    Vector2D position_temp = position + velocity * dt;

    // k2: aceleración estimada en el siguiente paso (suponiendo E constante)
    Vector2D acceleration_k2 = electricField * (charge / mass);

    // Promedio de pendientes
    velocity = velocity + (acceleration_k1 + acceleration_k2) * (dt / 2.0);
    position = position + (velocity_k1 + velocity_temp) * (dt / 2.0);
}
```

Para mayor precisión y eficiencia en los cálculos de distribuciones de carga mayores, se puede implementar un método de Runge-Kutta de segundo orden que corrige el método de Euler.

Archivo main.cpp

- La impresión de los resultados en consola puede ralentizar el cálculo, y como buscamos eficiencia, puede eliminarse para solo enviarlos a los archivos.
- Calcular un solo campo para luego utilizar este valor con el operado ‘-’ ya que por simetría del campo se obtiene el opuesto.

```
// Bucle de simulación
for (int i = 0; i < steps; ++i)
{
    // Calcula campo eléctrico en la posición de cada partícula
    Vector2D E1 = calcularCampo(p1, p2.position);
    Vector2D E2 = calcularCampo(p2, p1.position);

    // Actualiza posición/velocidad de las partículas ( $F = q \cdot E$ )
    p1.update(E2, dt);
    p2.update(E1, dt);

    // Muestra posiciones en consola
    std::cout << "Paso " << i << ":\n";
    std::cout << "  Partícula 1: " << p1.position << "\n";
    std::cout << "  Partícula 2: " << p2.position << "\n";

    // Guarda posiciones en archivos
    archivo1 << p1.position << "\n";
    archivo2 << p2.position << "\n";
}
```

```

# Compilación en modo Debug
debug:
    $(CXX) $(CXXFLAGS_DEBUG) $(SRC) -o $(TARGET_DEBUG)

# Compilación en modo Release
release:
    $(CXX) $(CXXFLAGS_RELEASE) $(SRC) -o $(TARGET_RELEASE)

# Limpieza
clean:
    rm -f *.o $(TARGET_DEBUG) $(TARGET_RELEASE) \
        particula1.txt particula2.txt \
        tiempos_debug.txt tiempos_release.txt \
        grafica_tiempos.png trayectorias.png temp_time.txt

run_debug:
    @if [ ! -f $(TARGET_DEBUG) ]; then \
        echo "Compilado Debug no encontrado. Compilando..."; \
        $(MAKE) debug; \
    fi
    ./${TARGET_DEBUG}

run_release:
    @if [ ! -f $(TARGET_RELEASE) ]; then \
        echo "Compilado Release no encontrado. Compilando..."; \
        $(MAKE) release; \
    fi
    ./${TARGET_RELEASE}

```

makefile

- Asignar diferentes archivos de compilación .o para debug y release.
- Aplicar parámetro de paralelización en el uso de los núcleos del procesador con la opción `make -j`.

¡Gracias!